Last Update: 8-21-2026

Course	CSC 4553 Intro to Operating Systems
Lecture	Section 17: Tuesday & Thursday, 12:00-12:50, Constr Innovations Center C102
Lab	Section 18: Tuesday & Thursday, 13:00-13:50, Engineering IV 0206
Instructor	Paul Schmitt
Office	In person: 14-204, Virtual: https://calpoly.zoom.us/my/paulschmitt
Office Hours	Tuesday & Thursday, 14:00-15:00, Virtual: By appointment
Email	prs@calpoly.edu, (please put "**[CSC 4553]**" in subject line)
Web Page	https://canvas.calpoly.edu/courses/187882

Course Description

The course will focus on the theory, practice, and technologies used to build and reason about modern operating systems. This course is a broad survey of the field, and its overarching goal is to give students a taste of the core principles that affect the design, evaluation or simply, the popular understanding of modern operating systems.

Textbooks

Operating Systems: Three Easy Pieces, Version 1.10, R.H Arpaci-Dusseau and A. C. Arpaci-Dusseau.

Prerequisites

Required: CPE/CSC 357 or CPE/CSC 2050; and CPE/CSC 225, CPE/EE 233, or CPE 2300.

Recommended: CSC 364 or CSC 3001; and CPE 316 or CPE 3160.

Learning Objectives

The course will examine the requirements for and structure of a modern operating system, including the fundamental problems of resource allocation and management of concurrent processes. By the end of the semester students will be expected to be able to:

1. Explain the basic components in a computer architecture, the subsystems within an operating system, and how they interoperate to form a general-purpose computer.
2. Assess and explain the benefits and pitfalls of multiprogramming and multi-threading, including concurrency, synchronization, and deadlock.
3. Explain the role the operating system plays in booting a system, resource management and virtualization, and process execution.
4. Contrast differing operating system paradigms, including monolithic, micro, and ring-based kernels.
5. Evaluate the tradeoffs between various operating system algorithms, such as those used for memory management and page replacement, disk scheduling, and process scheduling.
6. Implement basic techniques for managing concurrency, including critical sections, synchronization, and deadlock detection and avoidance.
7. Explain the operating system's role in securing a computer.
8. Apply "mechanical sympathy" and system-thinking skills to the construction of software that utilizes significant system resources.

Topics (subject to change)

The course runs 15 weeks. Weeks below overlap because most weeks split across two lecture sessions, and a unit frequently ends mid-week.

Weeks	**Topic**	**Subtopics**
1–2	Introduction, OS Structures & System Calls	What is an operating system? A brief history of OSes; OS abstractions and design tradeoffs; the OS as resource manager; the boot chain (firmware → bootloader → kernel → init/PID 1); kernel architectures (monolithic, layered, microkernel, hybrid); dual-mode operation, protection rings and privilege levels; interrupts, traps, and the system call interface
3–4	Processes & IPC	The process model, states, and context switching; the UNIX process API (`fork`/`exec`/`wait`), zombies and orphans; the process as

Weeks	Topic	Subtopics
		a protection boundary; modern isolation: namespaces, cgroups, containers; message passing vs. shared memory; pipes, FIFOs, `mmap`, POSIX vs. System V IPC
4–5	Scheduling	Preemptive vs. non-preemptive scheduling, metrics; batch algorithms (FCFS, SJF, SRTF, priority); interactive algorithms (Round Robin, multi-level queues, MLFQ), starvation and gaming the scheduler
5–6	Threads & the Thread API	Processes vs. threads; parallelism and Amdahl's Law; thread models (user/kernel, M:1, M:N, 1:1); the pthreads API
7–8	Synchronization & Deadlocks	Race conditions, critical-section requirements, hardware atomics; mutexes, spin vs. block, futexes, priority inversion; semaphores and condition variables; the four conditions for deadlock and resource-allocation graphs; prevention, avoidance, detection, and recovery
9–11	Memory & Virtual Memory	The memory hierarchy, base and bounds, address binding; swapping, contiguous allocation, and external fragmentation; paging and page tables; page

Weeks	Topic	Subtopics
		replacement (OPT, FIFO, LRU, clock, Belady's anomaly); TLBs and TLB reach; multilevel and inverted page tables; demand paging, working set, thrashing, huge pages; memory protection: ASLR, KASLR, DEP and the NX bit
12–13	File Systems	Files, directories, and their operations; file permissions and ownership; on-disk layout: superblocks, inodes, indirect blocks, free-block tracking; the Linux VFS and its three-table model; path resolution and mounting; crash consistency: `fsck`, journaling, ext3, log-structured file systems; backup superblocks and recovery
14–15	Storage	HDD geometry and organization; disk scheduling (FCFS, SSTF, the SCAN family); LVM; RAID 0/1/4/5; SSDs, NAND, and the FTL

Teaching Modalities

This course balances lectures with hands-on assignments and lab work. Lecture and lab each meet **twice a week, Tuesday and Thursday**, back to back. Both are opportunities for the instructor to present course material, although lecture is used primarily for instruction and lab primarily for guided practice and review.

Course Environment

Nearly all work in this course runs inside the **course Debian VM**, which you will set up in Lab 01 in the first week. Several labs and assignments require root privileges, loop devices, or kernel-module loading, so they cannot be completed on the department Unix servers. Set the

VM up early. A broken environment in Week 6 is not an extension-worthy excuse.

Programming assignments are graded on a Debian 13 x86-64 machine. This is the same environment as the course VM, so if it builds and runs there, it will build and run on the grader. That is the whole reason the VM exists, and it is why "it worked on my laptop" is not a defense. If you choose to develop somewhere else (macOS, WSL, your own distro, a container), that is fine, but **verify on the course VM before you submit**. Every assignment ships an autograder you can run yourself; run it there.

All C code is written to the **C99 standard**, and it must build cleanly with `-Wall -Wextra` (assignments will tell you if `-pedantic` is also required). *Programs that fail to compile or run on the grading machine receive no credit.*

Readings

The primary text is OSTEP, assigned by chapter alongside the weekly modules on Canvas. It is strongly suggested you read **before** the associated lecture.

Man pages are required reading, not supplementary. OSTEP has no chapter on IPC, containers, VFS structure, or the ext2 recovery toolchain, and this course covers all four. Where a week lists man pages on Canvas, they carry the same weight as the textbook and are fair game on quizzes.

Two references are worth reading early if your C is rusty: *Beej's Guide to C Programming* and *The Missing Semester of Your CS Education*. Both are linked in the Resources module on Canvas.

Quizzes, Labs & Programming Assignments

Coursework will consist of weekly online quizzes, lab exercises, and (at least) four programming assignments. All assignments will be posted on Canvas.

All work is expected to be done individually, unless you have received expressed written consent (not implied oral consent) by the instructor.

All homework and programming assignments will be submitted using Canvas. Written exercises are to be submitted in plain text with each response clearly attributed to the assigned problem. Repeating the problem statement is not required. Instructions for submitted programming assignments are provided in each assignment. NOTE: Be careful to submit your final version and to have tested it before submitting. Programs that fail to compile or run will receive no credit. We will use the C99 standard for all C code.

Live Code Reviews

Some of the programming assignments include a required one-on-one code review with

the instructor or a TA, scheduled shortly after the deadline. These are short, roughly 5 to 10 minutes.

Here is exactly how they work, so there are no surprises:

- You will not know in advance which part of your work you will be asked about. The grader picks.
- You run your own code, on your own machine, without notes.
- You may be asked to reproduce a result, explain what is happening at the OS level, or answer a follow-up, often *"change this and tell me what will happen before you run it."*

Missing a scheduled review without arranging an alternative in advance forfeits those points.

Late Policy

I try my best to give plenty of time for all assignments. That said, you are responsible for keeping track of posted assignments and their due date/time.

Late work is accepted, and it is automatically penalized 25% for each 24 hours it is late. The deduction is applied to the assignment's total possible points:

Submitted	**Penalty**	**Most you can earn**
On time	none	100
Up to 24 hours late	25%	75
24 to 48 hours late	50%	50
48 to 72 hours late	75%	25
More than 72 hours late	100%	0

The penalty starts the moment the deadline passes. One minute late and one hour late are both in the first 24-hour block, and both cost 25%. There is no grace period.

If something serious happens (illness, emergency), talk to me. That is a different conversation from the late penalty, and the earlier you start it the more I can do.

Email

When you email the instructor, please put the class name in the subject line (*i.e.*, [CSC 4553]). All email correspondence must adhere to academic and professional guidelines.

Exams

Three exams–two midterms and one final–will be given. All are in-class and **closed-book**. Makeup exams will only be given under **extreme** circumstances.

Dates

Exam	When
Midterm 1	**Week ~6**, in-lecture, Constr Innovations Center C102
Midterm 2	**Week ~11**, in-lecture, Constr Innovations Center C102
Final	**Thursday, December 17**, 13:00-15:30, Constr Innovations Center C102

The final is cumulative.

Grade Breakdown

Programming Assignments 30%

Quizzes	8%
Labs	12%
Midterm Exams	25%
Final Exam	25%

Letter Grades:

A:	>=93
A-:	>=90
B+:	>=87
B:	>=83
B-:	>=80
C+:	>=77
C:	>=73
C-:	>=70
D+:	>=67
D:	>=63
D-:	>=60
F:	<60

All graded work is compulsory. The instructor reserves the right to amend this breakdown at any time.

Conduct In Class

Please silence your cell phones and refrain from engaging in activities that might distract your fellow classmates (e.g., surfing the web). Attending lectures and labs are not compulsory, so if you don't want to be there, don't make it harder for someone who does.

Academic Integrity

Collaboration

All assignments in this class are intended to be demonstrations of individual or, in the case of a group assignment, partnership abilities. To this end, programs are to be written only by the designated author(s). High-level discussion of problems and problem-solving techniques, however, is beneficial to all involved. You are encouraged to discuss approaches so long as those with whom you consult are given due credit in your program headers.

It is never acceptable to allow someone else to use your work for reference while writing your own.

In this case, "someone else's work" means not only other students' programs, but also materials from any other source, including, but not limited to, the world wide web, other reference books, or previous course materials. Collaboration that goes beyond general approaches or that is uncredited will be considered cheating. If you are unsure about what constitutes proper or improper collaboration, consult the instructor for guidance.

Generative AI

You are welcome to use generative AI tools (such as ChatGPT, Google Gemini, Microsoft Copilot, Claude, etc.) in this class, and in some cases, you will be expected to do so. AI usage should be documented and cited. State which tools you used and what you used them for, such as debugging, explaining a man page, generating code, or writing prose. "None" is a complete answer, and an honest disclosure costs you nothing. You are responsible for respecting intellectual property and ensuring that any information generated by AI is accurate and ethical.

You own what you submit. If it is in your submission, you are claiming you understand it. The live code reviews exist because this course cannot detect AI use by inspecting your code, and will not try to. A student who used AI thoughtfully, understood the result, and can reason about it will do fine in a code review.

Cheating

Academic dishonesty is a serious offense, taken seriously. Any instances of cheating or plagiarism will be referred to the Office of Student Rights and Responsibilities. The Cal Poly rules and policies are listed in the catalog, as well as at the OSRR web site, http://www.osrr.calpoly.edu. My general policy, however, is very simply:

Cheating will result in an "F" course grade.

Turning in work is presumed to be a claim of authorship unless explicitly stated otherwise. If the course rules are unclear or you are unsure of how they apply, ask the instructor beforehand.

A Final Disclaimer

This document is a “living” document, and is subject to change at any time. If you have a question about the conduct of the course, please first refer to the “live” document, and not a copy. Any remaining unanswered questions should be directed to the instructor.